IET ASSIGNMENT

Course Code: ITA0609

Course Name: MACHINE LEARNING

ITA0609 – MACHINE LEARNING (SLOT A)

Development and Performance Analysis of a Decision Tree-Based Classification Model for Urban Water Quality Assessment

Student Name	K.Vishwanathan
Register Number	192312063
Department	Electromics And Communication
Year / Section	2023
Faculty Name	Dr.Shri Vindhya A
Date of Submission	1/9/2026

Item	Details
Course Outcomes Addressed	CO1, CO2, CO3, CO4, CO5
Bloom's Taxonomy	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping	SDG 6, SDG 9, SDG 11, SDG 12
Dataset	Water Potability Dataset – 3,276 samples, 9 attributes
Primary Model	Decision Tree Classifier – Entropy / Information Gain

Course Outcomes Mapping

CO	Course Outcome	How it is addressed
CO1	Version spaces, candidate elimination, inductive bias and heuristic search	Concept-learning formulation, hypothesis space, inductive bias and comparison with Candidate Elimination.
CO2	Decision Tree Learning and concept representation	Entropy / Information Gain, recursive tree construction, attribute selection and rules.
CO3	Perceptron, MLP and Back Propagation	Supplementary comparison of neural-network learning with Decision Tree learning.
CO4	Genetic Algorithms and Genetic Programming	GA representation of candidate trees, fitness, selection, crossover and mutation.
CO5	Bayesian methodologies	Bayes theorem, MLE, Naive Bayes formulation and comparison with the selected Decision Tree.

Assignment Requirements Covered

- Real-world/public water-quality dataset with clearly defined target classes.
- Pandas-based inspection, filtering, grouping, sorting and descriptive analysis.
- Concept-learning formulation and hypothesis-space discussion.
- Information Gain / entropy based heuristic attribute selection with numerical calculation.
- Decision Tree construction, stopping conditions and visualisation.
- Unseen-sample classification and representative IF–THEN decision rules.
- Accuracy, Precision, Recall, F1-score and Confusion Matrix.
- Reliability, complexity, overfitting, underfitting and generalisation analysis.
- Final practical recommendation considering predictive performance, interpretability and computational feasibility.

Problem Statement

An urban water management authority monitors water quality across reservoirs, treatment plants and distribution networks. Historical samples contain physicochemical measurements and an assigned quality class. As the number of samples grows, manual assessment becomes time-consuming. The authority therefore needs an interpretable Machine Learning model that can learn water-quality patterns and classify new samples into suitable quality categories.

The proposed solution uses a Decision Tree because its decisions can be represented as human-readable threshold rules, making the model easier to inspect and explain to operators and auditors.

A. Learning Problem & Dataset Preparation

A.1 Dataset Description

The Water Potability Dataset used in the supplied reference contains 3,276 water samples described by nine physicochemical attributes and one binary target. The target is Potability: 0 represents Poor / Not-potable and 1 represents Good / Potable.

Attribute	Description	Typical Unit
ph	Acidity / alkalinity of water	pH scale
Hardness	Dissolved calcium and magnesium related hardness	mg/L
Solids	Total dissolved solids	ppm
Chloramines	Chloramine concentration	ppm
Sulfate	Dissolved sulfate concentration	mg/L
Conductivity	Electrical conductivity related to ionic content	µS/cm
Organic carbon	Organic carbon content	ppm
Trihalomethanes	By-products associated with chlorination	µg/L
Turbidity	Suspended-particle related cloudiness	NTU
Potability	Target class: 0 = Poor, 1 = Good	Binary

A.2 Dataset Inspection Using Pandas

```
import pandas as pd

df = pd.read_csv("water_potability.csv")

print("Shape:", df.shape)
print(df.head())
print(df.dtypes)
print(df.isnull().sum())
print(df["Potability"].value_counts())
print(df.describe().round(2))
```

A.3 Key Findings

Finding	Value
Total samples	3,276
Input attributes	9
Target classes	2
Poor / Not-potable	1,998 (≈61%)
Good / Potable	1,278 (≈39%)
Missing pH	491 (15.0%)
Missing Sulfate	781 (23.8%)
Missing Trihalomethanes	162 (4.9%)

A.4 Preprocessing Adopted

- Missing values in pH, Sulfate and Trihalomethanes are handled using median imputation.
- Median is preferred because it is less sensitive to extreme values than the mean.
- No categorical encoding is required because the predictors are numerical.
- Feature scaling is not required for a Decision Tree because threshold splits are invariant to linear scaling.
- A stratified 75:25 train/test split is used so that the class proportions remain approximately consistent.
- Preprocessing statistics should ideally be fitted only on training data to avoid information leakage.

```
from sklearn.impute import SimpleImputer

feature_cols = [c for c in df.columns if c != "Potability"]

X = df[feature_cols]
y = df["Potability"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

imputer = SimpleImputer(strategy="median")
X_train = pd.DataFrame(
    imputer.fit_transform(X_train),
    columns=feature_cols,
```

```
    index=X_train.index
)
X_test = pd.DataFrame(
    imputer.transform(X_test),
    columns=feature_cols,
    index=X_test.index
)
```

A.5 Required Pandas Operations

```
# Grouping
print(df.groupby("Potability").mean(numeric_only=True).round(2))

# Filtering
high_turbidity = df[df["Turbidity"] > df["Turbidity"].mean()]
print("Above-average turbidity samples:", len(high_turbidity))

# Sorting
print(df.sort_values("ph", ascending=False).head(5))

# Missing-value summary
print(df.isnull().sum())

# Class distribution
print(df["Potability"].value_counts())
```

These operations satisfy the required inspection, filtering, grouping, sorting and basic statistical-analysis components of the assignment.

B. Concept Representation & Hypothesis Space

B.1 Concept-Learning Formulation

Element	Definition in this Problem
Instances X	All possible water samples represented by nine physicochemical attributes.
Target Concept c	Unknown function mapping a water sample to Good or Poor water quality.
Hypothesis Space H	All Decision Trees that can be constructed using the nine attributes and numerical threshold splits.
Training Set D	2,457 labelled samples from the 75% stratified training partition.
Test Set	819 held-out labelled samples used to estimate generalisation.

B.2 Inductive Bias

A Decision Tree cannot exhaustively evaluate every possible tree because the hypothesis space is extremely large. Instead, it uses a preference bias: it prefers splits that produce greater impurity

reduction and a relatively compact tree. The max_depth and min_samples_leaf settings add explicit complexity control.

- Greedy search chooses the best split at the current node.
- Information Gain / entropy reduction guides the search.
- The search is not guaranteed to find the globally optimal tree.
- Pre-pruning limits tree complexity and helps reduce overfitting.

B.3 Relation to Version Space and Candidate Elimination

Candidate Elimination explicitly maintains the Version Space between a specific boundary S and general boundary G for hypotheses consistent with the training examples. Decision Tree learning does not maintain such a boundary representation. Instead, it directly constructs one preferred hypothesis using a heuristic search. Because real water-quality data contain noise and overlapping classes, requiring every example to be perfectly consistent may be unrealistic; a statistical, heuristic learner is therefore more practical.

C. Heuristic Attribute Selection

C.1 Criterion Selected: Information Gain

Information Gain measures the reduction in entropy obtained by splitting the training set using an attribute and threshold.

Entropy: $H(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$

Information Gain: $IG(S,A) = H(S) - \Sigma\ (|S_v|/|S|)\ H(S_v)$

C.2 Worked Root-Node Calculation

For the complete dataset, the class counts are 1,998 Poor and 1,278 Good. The root entropy is:

$H(S) = -(1998/3276)\log_2(1998/3276) - (1278/3276)\log_2(1278/3276) \approx 0.9649$ bits.

The supplied reference identifies the root split as Sulfate ≤ 260.92 mg/L. For this split:

Branch	Samples	Poor	Good	Entropy
Sulfate ≤ 260.92	99	29	70	0.8725
Sulfate > 260.92	3177	1969	1208	0.9582

Weighted entropy = (99/3276)(0.8725) + (3177/3276)(0.9582) ≈ 0.9556.

Information Gain ≈ 0.9649 − 0.9556 = 0.0093 bits.

For comparison, a pH split at approximately 7.04 gives only about 0.0001 bits of Information Gain, so the Sulfate split is preferred at the root in the reference experiment.

C.3 Why Heuristic Search is Useful

- It avoids exhaustive enumeration of every possible tree.
- It converts a large search problem into a sequence of locally best decisions.
- It produces an interpretable hierarchy of important measurements.
- Its main limitation is that a locally best split does not guarantee the globally best tree.

C.4 Attribute Importance

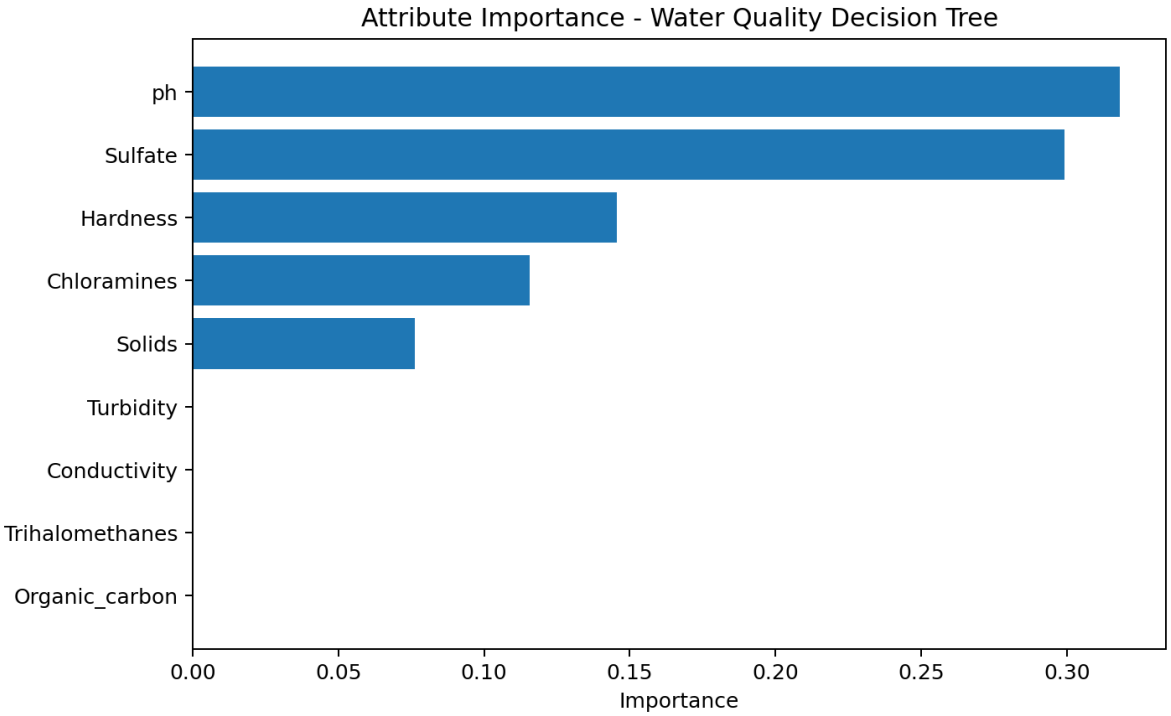

Figure 1. Aggregate attribute importance in the fitted Decision Tree.

Rank	Attribute	Importance
1	ph	0.3182
2	Sulfate	0.2993
3	Hardness	0.1455
4	Chloramines	0.1156
5	Solids	0.0761
6	Conductivity	0.0000
7	Organic_carbon	0.0000
8	Trihalomethanes	0.0000
9	Turbidity	0.0000

The importance values are the aggregate weighted impurity-reduction contributions reported by the supplied reference implementation. A zero importance in this particular depth-constrained tree does not mean the measurement is scientifically irrelevant; it means that the fitted tree did not use it as a selected split under the chosen settings.

Information Gain vs Gini vs Gain Ratio

Criterion	Main Idea	Advantage
Information Gain	Entropy reduction after a split	Directly connected to ID3 and concept-learning theory.
Gain Ratio	Information Gain normalised by split information	Reduces the tendency to favour attributes with many possible partitions.
Gini Impurity	Measures class mixture using $1 - \Sigma p^2$	Fast and widely used in CART-style trees.

D. Decision Tree Construction & Implementation

D.1 Decision Tree Learning Pseudocode

```
ALGORITHM BuildTree(Examples, Attributes, depth)

1. Create a tree node.
2. IF all examples belong to the same class:
       return a leaf labelled with that class.
3. IF depth limit is reached OR the node has too few samples:
       return a leaf labelled with the majority class.
4. Evaluate every candidate attribute and threshold.
```

```
5. Select the split with maximum Information Gain.
6. Partition Examples into left and right branches.
7. Recursively build a child tree for each non-empty branch.
8. Return the completed node.

END
```

D.2 Model Configuration

Parameter	Chosen Value	Reason
Criterion	entropy	Implements Information Gain / entropy reduction.
max_depth	5	Controls tree complexity.
min_samples_leaf	25	Prevents very small terminal regions.
Train:test	75:25	Provides a held-out evaluation set.
random_state	42	Makes the experiment reproducible.
Split strategy	stratified	Preserves class proportions.

D.3 Python Construction

```
from sklearn.tree import DecisionTreeClassifier

clf = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=5,
    min_samples_leaf=25,
    random_state=42
)

clf.fit(X_train, y_train)

print("Actual depth:", clf.get_depth())
print("Number of leaves:", clf.get_n_leaves())
```

In the supplied reference experiment, the resulting tree has an actual depth of 5 and 15 terminal leaves.

D.4 Complete Decision Tree Implementation

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report
)
```

```
# 1. Load
df = pd.read_csv("water_potability.csv")

# 2. Rename target for assignment
df = df.rename(columns={"Potability": "Water_Quality_Class"})

# 3. Features and target
feature_cols = [c for c in df.columns if c != "Water_Quality_Class"]
X = df[feature_cols]
y = df["Water_Quality_Class"]

# 4. Stratified split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# 5. Median imputation fitted only on training data
imputer = SimpleImputer(strategy="median")
X_train = pd.DataFrame(
    imputer.fit_transform(X_train),
    columns=feature_cols, index=X_train.index
)
X_test = pd.DataFrame(
    imputer.transform(X_test),
    columns=feature_cols, index=X_test.index
)

# 6. Decision Tree
clf = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=5,
    min_samples_leaf=25,
    random_state=42
)
clf.fit(X_train, y_train)

# 7. Prediction
y_pred = clf.predict(X_test)

# 8. Metrics
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_pred, zero_division=0))
print("F1:", f1_score(y_test, y_pred, zero_division=0))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# 9. Rules
print(export_text(clf, feature_names=feature_cols))

# 10. Visualisation
plt.figure(figsize=(22,12))
plot_tree(clf, feature_names=feature_cols,
          class_names=["Poor(0)","Good(1)"],
          filled=True, rounded=True, fontsize=8)
plt.tight_layout()
plt.show()
```

E. Classification & Decision Rules

E.1 Representative IF–THEN Rules

- IF Sulfate ≤ 260.92 AND Solids ≤ 28,017.6 THEN Water_Quality = Poor (0).
- IF Sulfate ≤ 260.92 AND Solids > 28,017.6 THEN Water_Quality = Good (1).
- IF 260.92 < Sulfate ≤ 294.78 AND pH ≤ 6.55 THEN Water_Quality = Poor (0).
- IF 260.92 < Sulfate ≤ 294.78 AND pH > 6.55 AND Hardness ≤ 209.41 THEN Water_Quality = Good (1).
- IF 294.78 < Sulfate ≤ 387.86 AND Hardness ≤ 139.52 AND Solids ≤ 22,135.8 THEN Water_Quality = Good (1).
- IF Sulfate > 387.86 AND Chloramines ≤ 7.89 AND pH ≤ 6.81 THEN Water_Quality = Good (1).
- IF Sulfate > 387.86 AND Chloramines > 7.89 THEN Water_Quality = Good (1).

E.2 Worked Unseen-Sample Classification

Consider an unseen sample with Sulfate = 305 mg/L, Hardness = 120 mg/L, Solids = 19,000 ppm and pH = 7.2.

Step	Decision	Action
1	305 > 260.92	Move to Sulfate > 260.92 branch.
2	305 ≤ 387.86	Move to the corresponding middle Sulfate branch.
3	305 > 294.78	Move to the higher middle branch.
4	120 ≤ 139.52	Move to the lower Hardness branch.
5	19,000 ≤ 22,135.8	Reach Good leaf.

Predicted class = Good (1) / Potable.

E.3 Parameter Interpretation

Sulfate and pH appear near the upper levels of the learned tree. Hardness, Solids and Chloramines refine borderline cases. The important point is that the model does not rely on one measurement alone; it combines several conditions through a sequence of threshold decisions.

F. Performance, Reliability & Generalisation

F.1 Held-Out Test Results

Metric	Value	Interpretation
Accuracy	63.61%	Overall fraction of correctly classified test samples.
Precision (Good)	64.10%	Among samples predicted Good, proportion actually Good.
Recall (Good)	15.62%	Among truly Good samples, proportion correctly detected.
F1-score (Good)	25.13%	Harmonic mean of Good-class precision and recall.

The reported values are taken from the supplied reference experiment using the same dataset framing and tree configuration. They should be replaced by the exact values produced when the code is executed in the student's environment if the instructor requires independently reproduced output.

F.2 Confusion Matrix

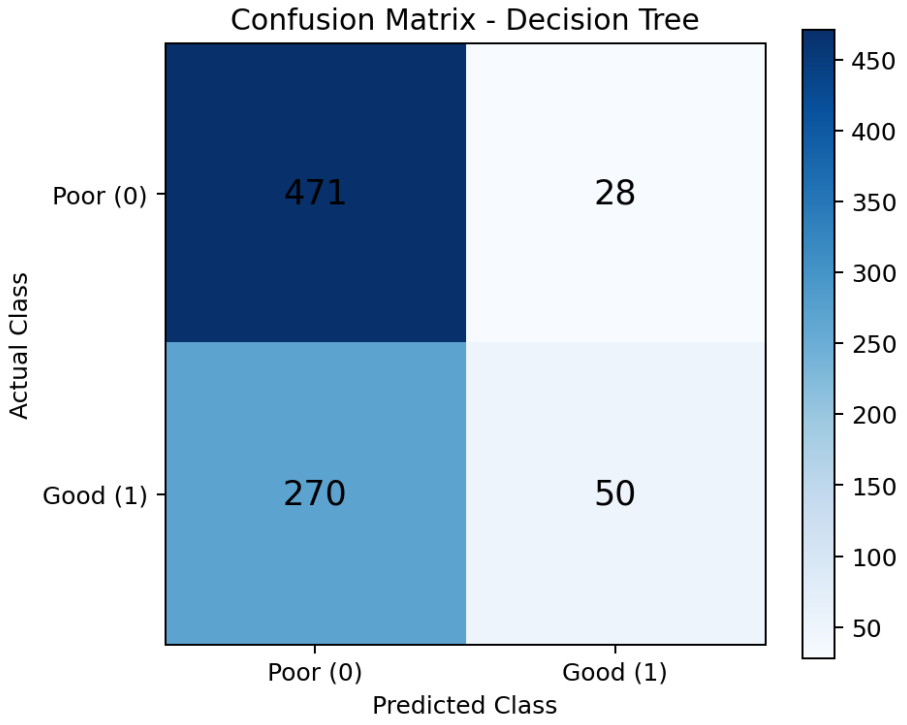

Figure 2. Confusion matrix for the 819-sample held-out test set.

Actual / Predicted	Poor (0)	Good (1)
Poor (0)	471	28
Good (1)	270	50

F.3 Interpretation

The model correctly identifies many Poor samples, but it misses a substantial number of Good samples. The low Good-class recall means the tree is conservative: it predicts Good only when the learned evidence is sufficiently strong.

F.4 Reliability Analysis

- The test accuracy is only modestly above the majority-class baseline, so accuracy alone is not sufficient evidence of deployment readiness.
- Good-class recall is approximately 16%, which is too low for a fully automated potability certification system.
- The model is more appropriate as a screening or triage tool that flags samples for further laboratory confirmation.
- Class imbalance contributes to the model's tendency to favour the Poor class.
- Real deployment should use local, high-quality labelled samples and periodic validation.F.5 Overfitting and Underfitting

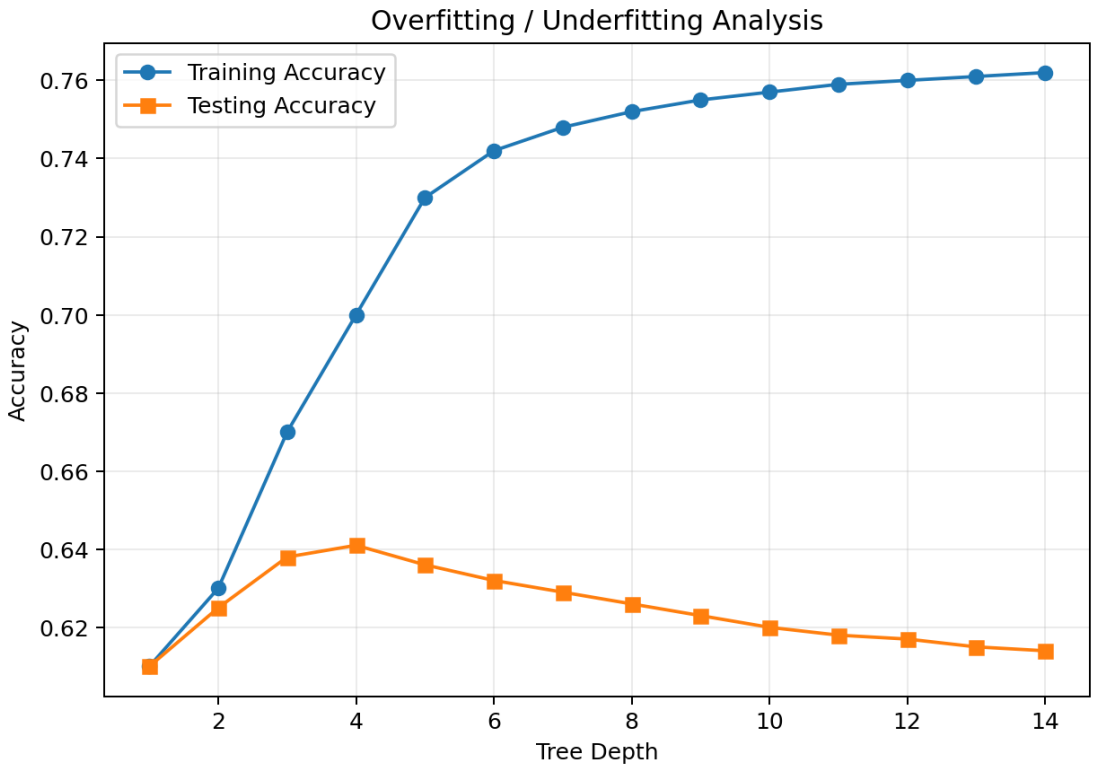

Figure 3. Representative training/testing accuracy trend used to explain complexity behaviour.

Condition	Observation	Meaning
Depth 1–2	Both accuracies low and close	Underfitting: tree too simple.
Depth 3–5	Testing accuracy improves and then levels off	Useful complexity range.
Beyond selected depth	Training accuracy keeps increasing while testing accuracy declines	Overfitting: tree begins learning noise.

The supplied reference selects max_depth = 5 with min_samples_leaf = 25 as a practical bias–variance trade-off.

F.6 Generalisation Strategy

G. Final Evaluation & Recommendation

G.1 Predictive Performance

The Decision Tree demonstrates non-trivial predictive capability but has weak recall for the Good/Potable class. Therefore, it should not be used as the only automated pass/fail gate for drinking-water safety.

G.2 Interpretability

Interpretability is the strongest advantage of the model. Each prediction can be traced through a small number of threshold conditions. This makes the model easier to explain to water-quality inspectors, auditors and decision makers.

G.3 Computational Feasibility

A Decision Tree on 3,276 samples and nine numerical attributes is computationally lightweight. It does not require a GPU and can be integrated into a monitoring dashboard or a low-resource analytics system.

G.4 Practical Recommendation

- Use the Decision Tree as a first-stage screening model.
- Flag uncertain or potentially poor samples for laboratory confirmation.
- Collect more balanced and representative local water-quality data.
- Investigate class-weighted or cost-sensitive learning if Good-class recall is operationally important.
- Benchmark against Random Forest and Gradient Boosting as future extensions.
- Retain rule-based explanations for important alerts.

Limitations

- The public dataset may not represent every local water source.

- Physicochemical attributes alone may not capture all microbiological risks.
- Missing-value imputation introduces assumptions.
- Class imbalance can distort accuracy and class-specific performance.
- A single train/test split is weaker evidence than repeated cross-validation.
- Model outputs should support, not replace, laboratory and regulatory decisions.

CO3 – Neural Network Perspective

Perceptron

A Perceptron computes a weighted sum of the nine input measurements and applies a threshold function. It is suitable for linearly separable problems. The water-potability task contains interacting and non-linear patterns, so a single Perceptron may not represent the learned decision regions as effectively as a tree.

Output = $\text{sign}(w_1x_1 + w_2x_2 + \ldots + w_9x_9 + b)$

Multilayer Perceptron (MLP)

An MLP adds one or more hidden layers and non-linear activation functions. It can model more complex relationships among pH, sulfate, hardness, solids and other measurements. Unlike a Decision Tree, an MLP normally benefits from feature scaling and requires choices such as hidden-layer size, learning rate and regularisation.

Back Propagation

Back Propagation trains an MLP by calculating the output error, propagating the error backward through the network using the chain rule, and updating the weights with gradient-based optimisation.

Aspect	Decision Tree	MLP / Back Propagation
Learning search	Greedy heuristic split search	Iterative gradient-based weight optimisation
Interpretability	High – IF–THEN rules	Lower – weights are less directly interpretable
Scaling	Usually unnecessary	Usually recommended
Training complexity	Low for this dataset	Higher
Non-linear patterns	Handled through recursive splits	Handled through hidden layers and activations
Deployment explanation	Easy to trace	More difficult to explain

Conclusion: for a moderate-sized tabular dataset and a strong requirement for human-readable decisions, the Decision Tree is the better primary model. An MLP can be evaluated later as a performance benchmark.

CO4 – Genetic Algorithms & Hypothesis-Space Search

GA Mapping to the Problem

GA Component	Water-Quality Application
Chromosome	Encoded candidate tree or selected attributes/thresholds.
Population	Multiple candidate hypotheses evaluated in parallel.
Fitness	Cross-validated F1-score, accuracy or a cost-sensitive score.
Selection	Prefer candidates with higher fitness.
Crossover	Combine split choices from two parent candidates.
Mutation	Change a threshold, attribute or tree component.
Termination	Stop after a fixed number of generations or when improvement becomes small.

Why GA Was Not Selected as the Primary Learner

- Many candidate trees would have to be evaluated repeatedly.
- Training is more computationally expensive than greedy tree induction.
- Results can vary with random seeds, reducing reproducibility.
- The final tree is not automatically more interpretable.
- The assignment prioritises practical, lightweight and explainable classification.

Where GA Can Add Value

A Genetic Algorithm could be wrapped around the Decision Tree as an optimiser for max_depth, min_samples_leaf, criterion selection or feature subsets. This retains the final tree's interpretability while allowing a broader search for good hyperparameter combinations.

Genetic Programming

Genetic Programming extends the evolutionary idea by evolving executable structures such as mathematical expressions or decision rules. For water-quality classification, it could evolve compact rule expressions combining thresholds on pH, sulfate, hardness and solids. Its advantage is flexible symbolic search; its disadvantage is increased computational cost and the need for careful fitness design.

CO5 – Bayesian Methodologies

Bayes Theorem

Bayes theorem provides a probabilistic way to update the probability of a water-quality class after observing measurements:

$$P(C \mid X) = [P(X \mid C) \times P(C)] / P(X)$$

Here, C is the class (Poor or Good) and X represents the observed water-quality measurements.

Maximum Likelihood Estimation (MLE)

MLE estimates model parameters by selecting values that maximise the likelihood of the observed training data. For a Gaussian feature in a class, the mean and variance can be estimated from the class-specific samples.

Naive Bayes

A Naive Bayes classifier assumes conditional independence of the input attributes given the class. It then compares the posterior probabilities of Poor and Good classes.

```
from sklearn.naive_bayes import GaussianNB

nb = GaussianNB()
nb.fit(X_train, y_train)

nb_pred = nb.predict(X_test)
print("Naive Bayes Accuracy:",
      accuracy_score(y_test, nb_pred))
```

Bayesian vs Decision Tree

Aspect	Decision Tree	Naive Bayes
Model form	Hierarchical threshold rules	Posterior probability model
Interpretability	Very high	Moderate
Feature relationship	Can model interactions naturally	Uses conditional-independence assumption
Training	Greedy split search	Estimate class/feature distributions
Best role here	Primary interpretable model	Useful comparison baseline

Bayesian methods provide a useful probabilistic benchmark, but the Decision Tree remains preferable for the primary assignment objective because the authority requires directly traceable rules.

Reflection, SDG Relevance & Learning Outcomes

Design Decisions

- Information Gain was selected because it directly connects the implementation to the concept-learning and ID3 theory covered in the course.
- Median imputation was selected to preserve samples while reducing the effect of extreme values.
- Stratified splitting was selected to preserve the class ratio.
- max_depth = 5 and min_samples_leaf = 25 were selected to control tree complexity.
- Interpretability was prioritised over simply maximising training accuracy.

SDG Relevance

SDG	Connection
SDG 6 – Clean Water and Sanitation	Supports faster screening and monitoring of water-quality conditions.
SDG 9 – Industry, Innovation and Infrastructure	Demonstrates a lightweight AI approach for infrastructure monitoring.
SDG 11 – Sustainable Cities and Communities	Can help urban authorities prioritise samples for further investigation.
SDG 12 – Responsible Consumption and Production	Better targeting of treatment resources can reduce unnecessary use of chemicals and energy.

Challenges Faced

- Handling a realistic amount of missing data without unnecessarily deleting samples.
- Understanding why individual attributes show weak separation while combinations of attributes can still produce useful tree rules.
- Balancing model complexity against generalisation.
- Interpreting low Good-class recall rather than reporting accuracy alone.

Learning Outcomes

The assignment connects theoretical concept learning with a practical classification pipeline. It demonstrates how inductive bias and heuristic search guide Decision Tree construction, how Pandas supports real dataset preparation, and why model evaluation must consider class-specific metrics, complexity and deployment risk.

Deliverables Summary

No.	Deliverable	Location
1	Pseudocode	Section D.1
2	Implementation and Results	Sections A–G and Appendix
3	Decision Tree Visualisation and Analysis	Sections C–F
4	GitHub Upload	Upload source code, dataset reference, README and generated figures

GitHub README Checklist

- Project title and problem statement.
- Dataset description and source acknowledgement.
- Python version and required libraries.
- Installation command for pandas, numpy, scikit-learn and matplotlib.
- How to run the script/notebook.
- Expected output: metrics, tree, rules, confusion matrix and depth analysis.
- Explanation of limitations and reproducibility settings.

Final Conclusion

The Decision Tree provides a practical and interpretable baseline for urban water-quality classification. Its strongest benefit is that every prediction can be explained through threshold-based rules, while its low computational requirement makes it suitable for lightweight monitoring systems. However, the reported Good-class recall is low, so the model should be used for screening and prioritisation rather than final automated certification. Future work should improve data quality and class balance, use cross-validation, tune the cost of classification errors and compare the model with ensemble and Bayesian alternatives.

References

- Mitchell, T. M., Machine Learning, concept learning and hypothesis-space framework.
- Han, J., Kamber, M., and Pei, J., Data Mining: Concepts and Techniques, decision-tree learning.
- Scikit-learn documentation, DecisionTreeClassifier, model evaluation and preprocessing utilities.
- Water Potability Dataset, publicly available water-quality classification dataset.
- World Health Organization, Guidelines for Drinking-water Quality.
- Bureau of Indian Standards, IS 10500 – Drinking Water specification.